

DESAFÍO 1: IMPLEMENTACIÓN SWEET CRUSH

Paulina Quintero Arenas, Luis Castillo Chicaiza

Facultad de Ingeniería Universidad de Antioquia
Informatica II
2026-02

1. CONTEXTUALIZACIÓN Y ANÁLISIS DEL PROBLEMA

Este desafío propone construir el juego Sweet Crush en C++ mediante la herramienta Qt. El problema central es la restricción para empaquetar la información del tablero, ya que no se pueden usar estructuras de datos para separarlos o clasificarlos para representar celdas individuales de bits. La idea es que el tablero completo se modele como una secuencia unidimensional y continua de bits gestionada por medio de la memoria dinámica heap. El almacenamiento de estos bits se propone por medio de bytes (8 bits). Recordemos que cada ficha usa 3 bits, por lo tanto, habrán situaciones donde una ficha puede estar contenida entre 2 bytes o en 1 solo.

1.1. Consideraciones de diseño

Para resolver este desafío, se definen los siguientes métodos para implementar:

- **Coordenadas a posición del bit:** Para obtener la ubicación física de cualquier ficha, se transforma su coordenada en un desplazamiento absoluto a nivel de bits

```
1 unsigned short bitInicial = ((fila *  
    columnas) + columna) * 3
```

Ya con este valor, se puede calcular el byte que contiene la ficha dividiendo entre 8

```
1 unsigned short byteIndice =  
    bitInicial/8
```

Y a partir de este, el desplazamiento interno, es decir, posición inicial de la ficha en ese byte usando el módulo

```
1 unsigned short offset = bitInicial % 8
```

- **Ventana para coordenadas:** Para manipular las fichas de interés de forma segura y sin alterar sus vecinos, se implementa una variable temporal de tipo *unsigned short* donde se almacena el byte actual y el siguiente con el fin de poder segmentar los 3 bits de interés por medio de desplazamientos y máscaras binarias.

```
1 unsigned short ventana = tablero[  
    byteIndice + 1]
```



Fig. 1. Ventana 16 bits

- **Mascara auxiliar para combinaciones:** Como se menciona en la guía, una ficha puede formar parte de una línea horizontal y vertical al mismo tiempo en una combinación, es por esto que cada que hayan combinaciones, estas no se deben eliminar al instante, sino que se debe hacer un mapeo completo del tablero para identificar si existen estos casos. Para esto, se plantea una mascara auxiliar la cual inicialmente está en cero y en caso de que esta sea similar a sus vecinos se activa en la mascara auxiliar que hay combinaciones. Una vez se hace todo el mapeo se eliminan las fichas, cuenta puntos y luego rellena.

Tablero fichas		
A	B	D
C	C	C
E	D	A
Mascara Auxiliar		
0	0	0
1	1	1
0	0	0

Fig. 2. Ejemplo mascara auxiliar

- **Gestión de memoria:** Se aplicó la regla de que si el

arreglo de bytes usados cae por debajo del 65 % se debe reducir el espacio del heap. Para esto, cada que se eliminen filas o columnas se vuelven a calcular los bytes usados y este valor se compara con el umbral, así puede proceder con la asignación menor en memoria si es necesario.

2. ARQUITECTURA DE SOLUCIÓN

Continuando con el propósito de que la practica sea modular, el sistema está constituido por 7 módulos con el fin de repartir los procedimientos que requiere el juego. A lo largo del código se podrá observar que la variable que más se utiliza es el *unsigned short* (2 bytes), esto con el fin de hacer uso adecuado de los recursos de la CPU ya que con esta variable es más que suficiente para la representación de algunos datos, sin necesidad de usar *int* que usa 4 bytes para su representación. La figura 3 muestra la arquitectura del software.

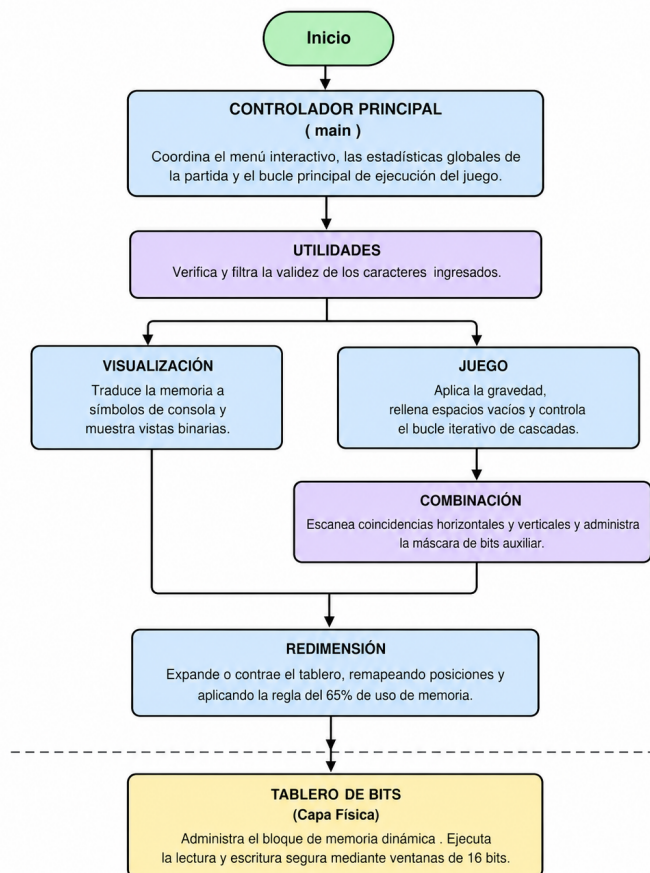


Fig. 3. Funciones del programa y su interacción

2.1. *tablero.cpp*

Esta librería es la encargada de reservar espacio en memoria y la manipulación de bits. Contiene las siguientes funciones:

- **bytesNecesarios():** Calcula la cantidad de bytes según las filas y columnas garantizando que ningún bit se quede por fuera. En esta función se calcula la cantidad de bits necesarios para el tablero, teniendo en cuenta que cada ficha contiene 3 bits.

```
1 unsigned short totalBits = filas *
   columns * 3;
```

Para retornar el valor de bytes necesarios sin que un bit de alguna ficha se quede por fuera, se .agrega un byte redondeando hacia arriba para garantizar que todos queden bien empaquetados

```
1 bytesNecesarios = (totalBits + 7) / 8
```

- **crearTablero():** Reserva el espacio en memoria dinámica necesario e inicializa el tablero en 0 para eliminar "basura". Esta función retorna un puntero de tipo *unsigned char** esto se hace por la necesidad de crear un espacio dinámico en memoria para las respectivas filas y columnas actuales:

```
1 unsigned char* tablero = new unsigned
   char[bytes]
```

Con esta declaración garantizamos que tablero sea una variable global y no desaparezca al finalizar la función ya que la necesitamos en el main() este tipo de declaración nos permite hacer redimensionamiento en tiempo real en cualquier otra función.

- **eliminarTablero():** Como la función anterior reserva un espacio de heap, este espacio se debe liberar al finalizar el código para evitar fugas de memoria.
- **leerFicha():** Transforma coordenadas (f,c) a posición de byte e índice en el byte para detectar el primer bit de la ficha de interés. Luego se pone en la ventana mencionada en la sección 1 y usando desplazamiento y la máscara 0x07 (0111) se obtienen los 3 bits de la ficha de interés.

Cuando se calcula que el bit inicial de la ficha es mayor a 5 significa que los 3 bits no están en el mismo byte sino que se comparte con el siguiente, es acá donde hace un desplazamiento hacia la izquierda y una OR para crear la ventana:

```
1 ventana |= (tablero[byteIdx + 1] << 8)
```

« 8 toma el byte vecino y lo mueve 8 posiciones hacia la izquierda colocandolo en la mitad izquierda (parte alta) de nuestra ventana de 16 bits. **OR** combina el byte original (que estaba en la mitad baja) con el nuevo byte desplazado. Luego se desplazan a la derecha la cantidad de offset y se hace una **AND** con 111

```
(ventana >> offset) & 0x07
```

Lo cual devuelve los 3 bits de la ficha.

- **escribirFicha():** Modifica el valor de la ficha de interés en el arreglo continuo de bits. Para esta función, se hacen las mismas operaciones de la anterior, para identificar los 3 bits. Luego, se deben eliminar estos bits (ficha vieja) para poder poner unos nuevos:

```
ventana &= ~ (0x07 << offset);
```

Usamos la misma mascara base 0x07 y la desplazamos el offset actual (para posicionarlos en los bits de interés). se invierten los bits. Se hace una **AND** y justo donde estaba la ficha se apaga estos bits y los demás se quedan intactos.

Ahora que tenemos el espacio vacío podemos proceder a poder un nuevo valor. Primero filtramos el nuevo valor, para asegurarnos que no traiga contenido basura en los 3 bits que nos interesan y con el offset nos ubicamos en la posición:

```
((valor & 0x07) << offset)
```

Luego devolvemos los bits a su byte correspondiente de la memoria del tablero:

```
tablero[byteIndice] = ventana & 0xFF
```

2.2. visualizacion.cpp

Módulo encargado de imprimir el tablero en la terminal, también tiene la opción de imprimir el tablero binario y la memoria, estas 2 fueron muy utiles al momento de hacer depuraciones y pruebas del código:

- **mostrarTableroSimbolos():** Encargado de ir ubicando los indices de filas, columnas y las respectivas fichas reales (A,B,C..) que tenga la memoria en esa posición.
- **mostrarTableroBinario():** Al igual que la anterior función ubica los indices, pero en vez de imprimir las fichas, imprime los bits correspondientes a las fichas.
- **mostrarMemoriaFisica():** Separa los bits de a 8 para armar los bytes correspondientes y así identificarlos con su respectivo índice para mostrarlo en memoria.

En esta librería las funciones `mostrarMemoriaFisica` y `mostrarTableroBinario` usan desplazamiento para poder imprimir el formato binario ya que c++ no lo interpreta así sino como un entero.

Para esto usamos el desplazamiento

```
(valor >> b) & 1
```

Con esto, estamos poniendo el bit en la posición del extremo derecho y al usar la mascara 1 obtenemos si el bit es 0 o 1. Lo unico que cambia en estas 2 funciones es la cantidad de bits que estamos leyendo.

2.3. juego.cpp

Esta función controla la dinámica del juego desde la generación de las fichas aleatorias, gravedad cuando hay vacíos y cascadas. Para este módulo fue necesario el uso de la librería `cstdlib` con el fin de poder generar números aleatorios de 1 a 6 correspondientes a las fichas.

En este módulo podemos encontrar que algunas funciones reciben otros punteros diferente a tablero, como lo son: `unsigned short* combinacionesTotales`, `unsigned int* fichasEliminadas`, o `unsigned int* puntuacion`, estas se declaran con el mismo fin de tablero, poder modificarlas globalmente ya que justamente estas son las estadísticas del juego y varias funciones pueden modificarlas.

- **tableroAleatorio():** Recorre el tablero filas a columnas y en cada posición genera y escribe una ficha aleatoria.
- **rellenarTablero():** Recorre el tablero en busca de fichas vacías "000" y las rellena con una nueva aleatoria.
- **aplicarGravedad():** Ya que la gravedad es caída de fichas, unicamente se revisan las columnas de abajo del tablero hacia arriba para identificar vacíos y con esto desplazar las fichas de arriba hacia abajo hasta que no se encuentre ningún vacío.
- **cascadas():** Controla las reacciones en cadena de las combinaciones, esto lo hace mediante la mascara auxiliar, luego borra los elementos marcados, aplica gravedad y rellena los espacios vacíos. Este proceso se repite hasta que ya no surgen más combos.
- **eliminarFichaManual():** Reemplaza la ficha elegida con un vacío "000", actualiza estadísticas de eliminaciones, aplica gravedad y rellena.

2.4. combinacion.cpp

Realiza el escaneo continuo para detectar combinaciones de 3 o mas fichas, calcula y acumula la puntuación actual.

- **crearMascara():** Reserva memoria dinámica para la máscara auxiliar para registrar qué fichas se eliminarán sin alterar el tablero real todavía.
- **destruirMascara():** Libera la memoria dinámica que había sido asignada al arreglo de la máscara.
- **marcarPosicion():** Modifica la máscara poniendo un bit específico en 1 para indicar que la coordenada de esa ficha pertenece a una combinación válida y debe ser destruida.
- **estaMarcada():** Lee el bit correspondiente en la máscara auxiliar para comprobar si la posición fue registrada previamente para eliminación.
- **detectarCombinaciones():** Escanea el tablero en ambos sentidos (horizontal y vertical) en busca de secuencias de 3 o más fichas vecinas iguales. Al hallarlas, las registra en la máscara y calcula los puntos obtenidos multiplicados por el nivel de cascada actual.
- **eliminarMarcados():** Recorre el tablero completo y asigna el estado de espacio libre 0 a todas las coordenadas que resultaron marcadas en la máscara, retornando el total de fichas destruidas.

2.5. *redimension.cpp*

Esta librería entra en función al momento de agregar/eliminar alguna ficha a columna, adicionalmente maneja la optimización del 65 % de memoria.

- **agregarFila():** Calcula y reserva la nueva memoria requerida, reubica las fichas desplazando hacia abajo las correspondientes, genera fichas aleatorias para el espacio insertado y elimina el arreglo antiguo.
- **eliminarFila():** Transfiere las fichas a un nuevo arreglo sin la fila seleccionada y evalúa la regla del 65 %; si la memoria en uso resulta menor a ese umbral, asigna un arreglo más pequeño para optimizar el espacio físico.
- **agregarColumna():** Reserva memoria para la nueva columna, copia cada ficha recalculando sus coordenadas para desplazar las columnas a la derecha de la posición de inserción, y rellena los huecos con fichas nuevas.
- **eliminarColumna():** Copia los datos del tablero saltándose la columna indicada para borrarla y aplica la validación estricta del 65 % de capacidad para decidir si compacta el tamaño del arreglo dinámico.

2.6. *utils.cpp*

Ya que al momento de jugar se requiere que el usuario ingrese valores numéricos, se crea esta función para no sobrecargar el main con validaciones. Esta librería se encarga de detectar que si se ingresen los caracteres adecuados según el caso:

- **Caso 0:** Para el ingreso de filas y columnas al momento de iniciar el juego, este número debe ser mayor o igual a 3.
- **caso 1:** Para manejar números de filas y columnas para las diferentes opciones del menú.
- **caso 2:** Maneja los números del menú de interacción, únicamente del 0 al 7.

2.7. *main.cpp*

El main despliega el menú de usuario y administra el bucle principal de ejecución mediante el llamado de las diferentes funciones para los casos necesarios.

Para la parte del juego, se implementa un while y dentro de este un switch case que va evaluando cada valor ingresado por teclado y dependiendo de este redirige las acciones.

Cuando el usuario desea finalizar el juego, se libera el espacio de memoria dinámica reservado durante el juego.

Las acciones que se despliegan en el menú para interacción con el usuario son las siguientes:

- 1 Eliminar ficha manual
- 2 Agregar fila
- 3 Eliminar fila
- 4 Agregar columna
- 5 Eliminar columna
- 6 Mostrar tablero binario
- 7 Mostrar memoria
- 0 Salir

2.8. Problemas de desarrollo afrontados y evolución de solución

- Una de las librerías más difícil de implementar fue la del tablero, no por el hecho de los desplazamientos sino más por el entendimiento previo de cómo se organizan los datos en memoria al momento de almacenarlos y con esto poder proponer los desplazamientos necesarios. Como el ser humano está acostumbrado a leer de izquierda a derecha creíamos que los bits de los bytes estaban organizados así [0 1 2 3 4 5 6 7 8] cuando en realidad están en el sentido contrario [7 6 5 4 3 2 1 0]

por eso es que cuando un bit se encuentra segmentado en 2 bytes se debe poner el bit siguiente adelante para lograr reorganizar adecuadamente.

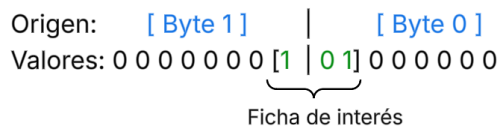


Fig. 4. Demostración de reorganización de bytes para lograr la ficha corecta

- Inicialmente habíamos planteado la lógica de que cuando se detectara un combo se eliminara inmediatamente, esto causaba que durante el barrido horizontal no consideraba las combinaciones verticales y posibles combinaciones en L y T. Para esto se plantea la mascara de detección de combinaciones, que hace todo el barrido para detectar combinaciones y luego elimina.
- La adición o eliminación de filas/columnas desajustaba los indices de todas las fichas posteriores. Para esto fue necesario reubicar todo el arreglo en un nuevo bloque de memoria dinámica sin perder ninguno.
- Para el pre- informe no consideramos el módulo *utils()*. Nos dimos cuenta de su importancia al momento de realizar el *main()* que integraba toda la lógica del juego. Ya que como se menciona anteriormente, se requiere validar que el valor ingresado por el usuario si sea adecuado. Este módulo nos facilitó el desarrollo del *main* e incluso lo dejó más limpio que haber tenido que hacer la comparaciones allí.

2.9. Representación de datos de bits

Para la codificación de los bits, cada casilla del tablero ocupa únicamente 3 bits. Este número de bits nos da un total de 8 combinaciones y unicamente son 6 fichas, entonces por facilidad definimos las fichas de la siguiente manera:

- **001:** A
- **010:** B
- **011:** C
- **100:** D
- **101:** E
- **110:** F

La combinación 000 se utilizó para indicar espacios vacíos. Se tomó la decisión de no utilizar la combinación 111.

2.10. Link del repositorio

https://github.com/LuisC645/Desafio_1.git

2.11. Link del Video

<https://youtu.be/9xZ9cqjPfY0>